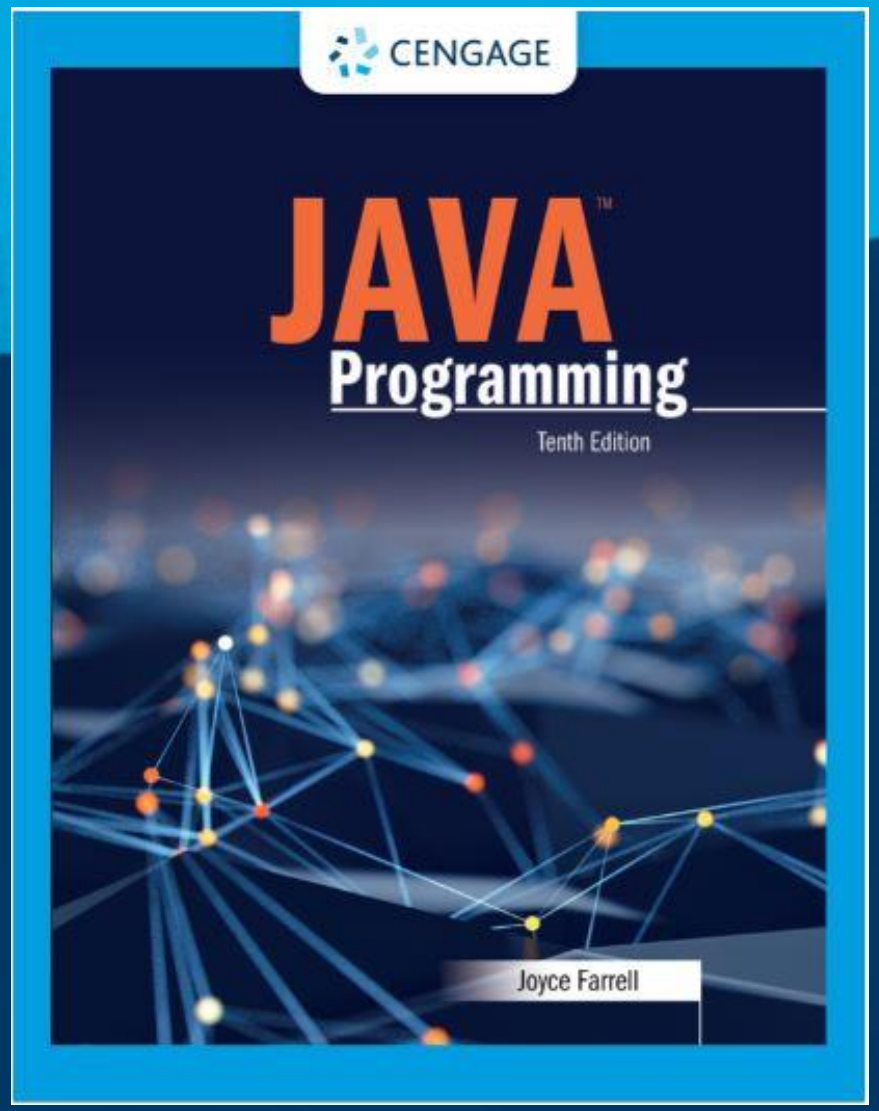


Java Programming, 10e

Chapter 10: Exception Handling

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Chapter Objectives

By the end of this chapter, you should be able to:

10.1 Describe exceptions

10.2 Try code and catch exceptions

10.3 Throw and catch multiple exceptions

10.4 Use the `finally` block

10.5 Appreciate the advantages of exception handling

10.6 Specify the exceptions that a method can throw

10.7 Trace exceptions through the call stack

10.8 Create your own `Exception` classes

10.9 Use an assertion

10.10 Provide a virtual keyboard for the user

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.1 Learning About Exceptions (1 of 4)

Exception

- An unexpected or error condition

Exception handling

- Object-oriented techniques that manage or resolve exceptions

Runtime exception

- An unplanned exception during the program's execution
  - Contrast with syntax error during compilation

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.1 Learning About Exceptions (2 of 4)

`Error` class

- Represents serious errors from which a program cannot recover
  - Insufficient memory to execute

`Exception` class

- Less serious errors from which the program can recover
- Unusual conditions that arise while the program is running
  - Invalid array subscript value
  - Program could recover by assigning a valid value

Crash

- When a program ends prematurely with an error

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.1 Learning About Exceptions (3 of 4)

Stack trace

- History list of error messages after each attempted execution

Mission critical

- A process that is crucial

Fault-tolerant

- Applications that continue to operate even if system fails

Robustness

- The degree to which a system is resilient to stress and able to continue functioning

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.1 Learning About Exceptions (4 of 4)

```
Enter numerator >> 12
Enter denominator >> three
Exception in thread "main" java.util.InputMismatchException
        at java.base/java.util.Scanner.throwFor(Scanner.java:860)
        at java.base/java.util.Scanner.next(Scanner.java:1497)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2161)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2115)
        at Division.main(Division.java:11)
```

```
Enter numerator >> 12
Enter denominator >> 3.0
Exception in thread "main" java.util.InputMismatchException
        at java.base/java.util.Scanner.throwFor(Scanner.java:860)
        at java.base/java.util.Scanner.next(Scanner.java:1497)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2161)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2115)
        at Division.main(Division.java:11)
```

Figure 10-4: Two executions of the `Division` application in which the user enters noninteger values

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 10.1: Knowledge Check

_____ represents the degree to which a system is resilient to stress and able to continue functioning.

a. Sturdiness

b. Robustness

c. Tolerance

d. Vigorousness

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 10.1: Knowledge Check Answer

_____ represents the degree to which a system is resilient to stress and able to continue functioning.

Answer: b. Robustness

Programs that can handle exceptions appropriately are said to be more fault tolerant and robust. Fault-tolerant applications are designed so that they continue to operate, possibly at a reduced level, when some part of the system fails. Robustness represents the degree to which a system is resilient to stress and able to continue functioning.

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.2 Trying Code and Catching Exceptions (1 of 4)

`try` block

- A block of code you attempt to execute knowing an exception might occur
- Try to “catch the exception”
- Consists of:
  - The keyword `try` followed by a pair of curly braces
  - Executable statements between the braces
  - Including some statements that may cause exceptions

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.2 Trying Code and Catching Exceptions (2 of 4)

`catch` block

- Programmed immediately after a `try` block
- Segment of code that can handle an exception thrown by the `try` block

`throw` statement

- Sends an `Exception` object out of a block or method so it can be handled elsewhere
- Each `catch` block can catch one type of exception
  - One object that is an object of type `Exception` or one of its child classes

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.2 Trying Code and Catching Exceptions (3 of 4)

To create a `catch` block:

- The keyword `catch` followed by a pair of parentheses
- Between the parentheses, an `Exception` type and an identifier for an instance
- A pair of curly braces that contain statements that take actions to handle the error condition

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.2 Trying Code and Catching Exceptions (4 of 4)

Making programs "foolproof"

- A `try` block is used to circumvent data entry errors
- `nextLine()` call to correct the problem

`try…catch` blocks

- A variable declared within one of the blocks goes out of scope when the block ends
- To use a variable with both a `try` or `catch` block and afterwards:
  - Declare the variable before the block begins
  - Be careful when assigning the value within the block

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.3 Throwing and Catching Multiple Exceptions (1 of 2)

With multiple `try` statements:

- You can place as many statements as you need in a `try` block and catch as many exceptions as you want
- If you try more than one statement, only the first error-generating statement throws an exception
- The rest of the statements are unexecuted

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.3 Throwing and Catching Multiple Exceptions (2 of 2)

With multiple `catch` blocks:

- Examined in sequence until a match is found
- The matching `catch` block executes
- Be careful some `catch` blocks don't become unreachable
- You can execute the same code no matter with `Exception` type occurs

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 10.2: Discussion

1. What are the purposes of `try` and `catch` blocks?
2. Describe what happens in an unreachable statement.

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 10.2: Discussion Debrief

1. A `try` block is a block of code you attempt to execute while acknowledging that an exception might occur. A `catch` block is a segment of code that can handle an exception that might be thrown by the `try` block that precedes it.
2. An unreachable statement is a program statement that can never execute under any circumstances, such as if one block is in the way of another. You can arrange your `catch` blocks so that the "bigger basket" is always below a smaller one.

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.4 Using the `finally` Block (1 of 2)

`finally` block

- Used when you have actions you must perform at the end of a `try…catch` sequence
- Code executes whether or not the preceding `try` block identifies an `Exception`
- Example: When the program uses data files that must be closed

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.4 Using the `finally` Block (2 of 2)

```
try
{
   // statements to try
}
catch(Exception e)
{
   // actions that occur if exception was thrown
}
finally
{
   // actions that occur whether catch block executed or not
}
```

Figure 10-24: Format of `try...catch...finally` sequence

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 10.3: Knowledge Check

The code in a _____ block executes regardless of whether the preceding `try` block identifies an `Exception`.

a. `certainly`

b. `definitely`

c. `absolutely`

d. `finally`

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 10.3: Knowledge Check Answer

The code in a _____ block executes regardless of whether the preceding `try` block identifies an `Exception`.

Answer: d. `finally`

When you have actions you must perform at the end of a `try...catch` sequence, you can use a `finally` block. The code within a `finally` block executes regardless of whether the preceding `try` block identifies an `Exception`. Usually, you use a `finally` block to perform cleanup tasks that must happen regardless of whether any `Exceptions` occurred and whether any `Exceptions` that occurred were caught.

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.5 Understanding the Advantages of Exception Handling (1 of 3)

Java's error-handling technique

- `try…catch` object-oriented technique
- Groups statements that do the "real work" together
- `try` steps usually work without generating errors
- Exceptional events are grouped and moved out of the primary action

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.5 Understanding the Advantages of Exception Handling (2 of 3)

```
try
{
    call methodA() and maybe throw an exception
    call methodB() and maybe throw an exception
    call methodC() and maybe throw an exception
    everything's okay, so display finalResult
}
catch(methodA()'s error)
{
   set errorCode to "A"
}
catch(methodB()'s error)
{
   set errorCode to "B"
}
catch(methodC()'s error)
{
   set errorCode to "C"
}
```

Figure 10-27: Pseudocode representing object-oriented exception handling

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.5 Understanding the Advantages of Exception Handling (3 of 3)

Object-oriented exception handling advantages

- Clarity
- Flexibility in handling error situations
  - In most object-oriented programs that same method that throws an exception doesn't catch it
- Can be reused

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.6 Specifying the Exceptions That a Method Can Throw (1 of 5)

`throws` clause

- Used if a method throws an exception that a different method will catch
- Exception specification
- Needed for checked exceptions

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.6 Specifying the Exceptions That a Method Can Throw (2 of 5)

Java exception types

- Unchecked
  - Inherit from the `Error` or `RuntimeException` class
  - Not required to handle in your program
- Checked
  - Should be anticipated by programmers
  - Program should be able to cover

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.6 Specifying the Exceptions That a Method Can Throw (3 of 5)

Catch or specify requirement

- If you throw a checked exception from a method:
  - Catch it within the method
  - Specify the exception in the method header's `throws` clause
- If you write a method with the `throws` clause in the header, any method using the method must:
  - Catch and handle the possible exception
  - Declare the exception in its `throws` clause

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.6 Specifying the Exceptions That a Method Can Throw (4 of 5)

Syntatic sugar

- Aspects of a computer language that make it sweeter
- Easier to use
- `import.java.lang`
- Salt is feature that makes it harder to write bad code

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.6 Specifying the Exceptions That a Method Can Throw (5 of 5)

To use a method to its full potential:

- You must know its name
- Return type
  - Unless you don't want to use the value it returns
- Type and number of required arguments
- Type and number of exceptions it throws
- Separate multiple potential exceptions in method header with commas or specify more than one general parent class

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 10.4: Discussion

1. What are the advantages to object-oriented exception handling?
2. What is meant by the “catch or specify requirement?”

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 10.4: Discussion Debrief

1. One advantage is clarity. Another is the flexibility it allows in the handling of error situations. You gain the ability to appropriately deal with exceptions as you decide how to handle them. When you write a method, it can call another that might throw an exception, your method can catch the thrown exception, and you can decide what you want to do. Methods are also reusable.
2. Java programmers say that checked exceptions are subject to the catch or specify requirement, which means if you throw a checked exception from a method, you must do one of the following: Catch it within the method; or Specify the exception in your method header's `throws` clause.

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.7 Tracing Exceptions Through the Call Stack (1 of 3)

Call stack

- Memory location
- Computer-stored list of memory locations to which the system must return when methods end
- Execution stack, memory stack, or stacks
- When a method throws an exception and the method doesn't catch it, it's thrown to the net method up the call stack
  - The method that called the offending method

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.7 Tracing Exceptions Through the Call Stack (2 of 3)

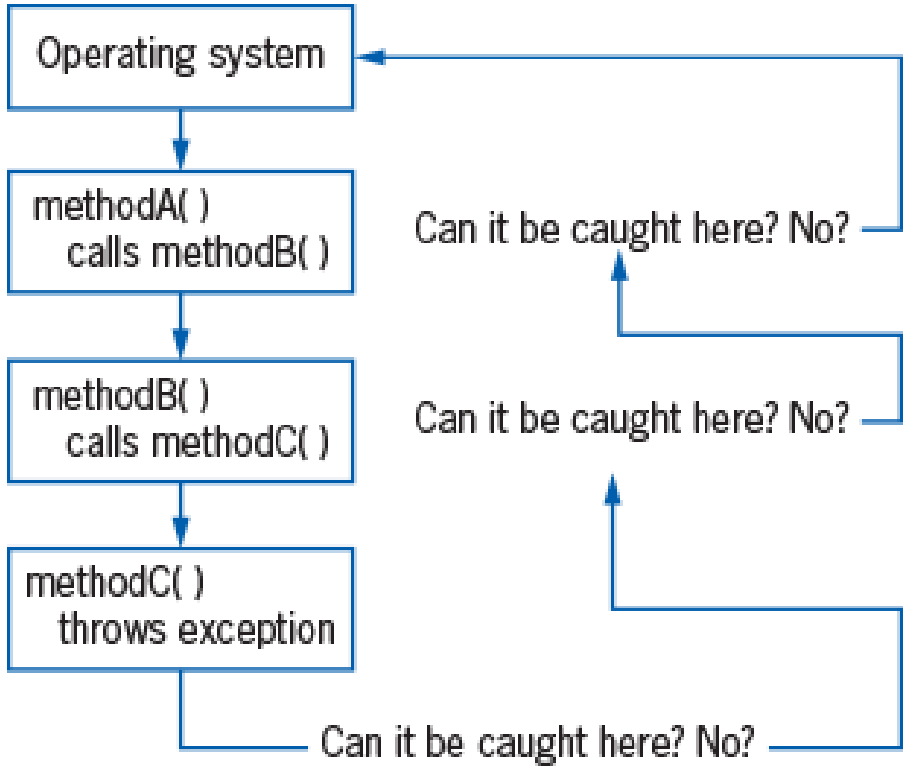


Figure 10-32: Cycling through the call stack

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.7 Tracing Exceptions Through the Call Stack (3 of 3)

`printStackTrack()` method

- `Exception` method
- Displays a list of methods in the call stack
- Determine the location of the statement that caused the exception

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.8 Creating Your Own `Exception` Classes (1 of 2)

Creating a throwable `Extension` class:

- Extend the `Exception` subclass of `Throwable`

Exception class constructors

- `Exception()`: new object with `null` detail message
- `Exception(String message)`: new object with specified message
- `Exception(String message, Throwable cause)`: new object with specified message and cause
- `Exception (Throwable cause)`: new objects with specified cause and detail message of `cause.toString()`

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.8 Creating Your Own `Exception` Classes (2 of 2)

Consider creating a catalog of possible messages

- All messages stored in one location
- List of possible errors is documentation of potential problems
- Other applications can use same catalog of messages
- Can provide messages in multiple languages

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.9 Using Assertions (1 of 2)

Using assertions

- Left in place unlike extra display statements that are found during programming
- If user does not use the `-ea` option the user will see no evidence the `assert` statements exist
- Reduce development and debugging time
- Don’t use to check for every type of possible error

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.9 Using Assertions (2 of 2)

```
public static boolean isEven(int number)
{
   boolean result;
   if(number % 2 == 1)
      result = false;
   else
   {
      result = true;
      assert number % 2 == 0 : number + " % 2 is " + number % 2;
   }
   return result;
}
```

Figure 10-43: The flawed `isEven()` method with an assertion

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.10 Displaying the Virtual Keyboard (1 of 3)

Virtual keyboard

- A computer keyboard that appears on the screen
- User operates by touch screen or using a mouse to point to and click keys
- `VirtualKeyboardDemo`

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.10 Displaying the Virtual Keyboard (2 of 3)

```
import java.util.Scanner;
import java.io.IOException;
public class VirtualKeyboardDemo
{
   public static void main(String[] args)
   {
      Scanner input = new Scanner(System.in);
      try
      {
         Process proc = Runtime.getRuntime().exec
            ("cmd /c C:\\Windows\\System32\\osk.exe");
      }
      catch(IOException e)
      {
         System.out.println(e.getMessage());
      }
      String name;
      System.out.print("Enter name >> ");
      name = input.nextLine();
      System.out.println("Hello, " + name + "!");
   }
}
```

Figure 10-48: The `VirtualKeyboardDemo` application

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

10.10 Displaying the Virtual Keyboard (3 of 3)



Figure 10-49: The `VirtualKeyboardDemo` program during execution

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 10.5: Case Study

You have been tasked with writing a program that uses exceptions to anticipate and deal with potential errors.

- What is the purpose of a call stack? What is the advantage of using the stack?
- What is the advantage of using an `Exception` class?
- Should you use an assertion to check for every type of error that could occur? Why or why not?
- What does the acronym *osk* stand for?

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 10.5: Case Study Debrief

- The call stack is the memory location where the computer stores the list of memory locations to which the system must return when methods end. Cycling through the methods in the stack allows methods to handle exceptions wherever the programmer has decided is most appropriate.
- Specialized `Exception` classes enable you to separate your error code from the usual, nonexceptional sequence of events; they allow errors to be passed up the stack and traced; and they allow clients of your classes to handle exception situations in the most suitable way.
- You do not want to use assertions to check for every type of error that could occur in a program. Assertions are meant to be helpful in the development stage of a program, not when it is in production and in the hands of users.
- *osk* stands for *on-screen keyboard*.

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Don’t Do It

- Don’t forget that all the statements in a `try` block might not execute.
- Don’t forget that you might need a `nextLine()` method call after an attempt to read numeric data from the keyboard throws an exception.
- Don’t forget that a variable declared in a `try` block goes out of scope at the end of the block.
- Don’t forget that when a variable gets its usable value within a `try` block, you must ensure that it has a valid value before attempting to use it.
- Don’t forget to place more specific `catch` blocks before more general ones.
- Don’t forget to write a `throws` clause for a method that throws a checked exception but does not handle it.
- Don’t forget to handle any checked exception thrown to your method either by writing a catch block or by listing it in your method’s `throws` clause.

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Self-Assessment (1 of 2)

1. What are the two basic classes of errors?
2. What is the purpose of a `throw` block?
3. Can a method throw any number of `Exception` types?
4. What is the advantage to using a `finally` block over entering repetitious code?
5. Why is object-oriented exception handling better than previous methods?

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Self-Assessment (2 of 2)

6. What are the two types of Java's exceptions?
7. List other names for call stack.
8. List and describe the four constructors in the `Exception` class.
9. What is the syntax of an `assert` statement?
10. Explain how the user interacts with a virtual keyboard.

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Summary

Click the link to review the objectives for this presentation.

Link to Objectives

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.